

Auto Judge

By Aarav Singh

24116003

Introduction:

Online competitive programming platforms classify problems into difficulty levels such as easy, medium and hard. AutoJudge tries to replicate that through a machine learning-based system that automatically predicts the difficulty level and numerical difficulty score of programming problems using only textual information.

The system uses classical machine learning techniques including TF-IDF-based text representation and some hard-coded features. Separate models are trained for classification and regression tasks. The trained models are evaluated using accuracy, confusion matrix, MAE, and RMSE metrics. A Flask-based web interface allows the user to give a problem statement and immediately check its difficulty level and score.

Dataset Used:

The dataset used in this project is available at <https://github.com/AREEG94FAHAD/TaskComplexityEval-24>

80% was used for training, and 20% for testing.

Each line corresponds to a JSON object, which has the following fields:

- Title
- Problem description
- Input description
- Output description
- Difficulty class (Easy / Medium / Hard)
- Numerical difficulty score

Preprocessing

Before model training, the textual data was pre-processed to reduce noise (removed extra spaces, all links, converted to lowercase and handled cases where input is not a string). Also, all textual fields were concatenated into a single field named `full_text`.

```
4 def clean_text(text) :
5     if not isinstance(text, str):
6         return ""
7
8     text = text.lower()
9     text = re.sub(r"http\S+|www\S+", " ", text)
10    text = text.replace("\n", " ")
11    text = text.replace("\t", " ")
12    text = text.replace("\r", " ")
13    text = re.sub(r"\s+", " ", text)
14    text = text.strip()
15
16    return text
17
18
19 def preprocess_text(path):
20     df = pd.read_json(path, lines=True)
21
22     df["full_text"] = (
23         df["title"].fillna("") + " " +
24         df["description"].fillna("") + " " +
25         df["input_description"].fillna("") + " " +
26         df["output_description"].fillna("")
27     )
28     df["full_text"] = df["full_text"].apply(clean_text)
29     return df
30
```

Feature Engineering

a) TF-IDF Features:

TF-IDF (Term Frequency–Inverse Document Frequency) was used to convert textual data into numerical form. Unigrams and bigrams were extracted to capture both individual keywords and common phrases related to programming concepts.

b) Hard-coded Features:

These include text length, frequency of mathematical and bitwise symbols, mathematical concepts like mod, gcd, lcm and occurrence of algorithm-specific keywords such as graph, tree and subsequence (Didn't use dp because it did not appear in training dataset even once).

Another feature introduced was constraints. Usually if the constraints are small (say $n < 1e3$), then an $O(n^2)$ solution is expected, making the problem easier as opposed to one with constraint $n < 1e5$.

```
def constraint_features(text):
    text = text.lower()

    features = 0
    power_constraints = len(re.findall(r"10^\d+", text))
    features += power_constraints

    large_numbers = len([int(x) for x in re.findall(r"\b\d+\b", text) if int(x) >= 100000])
    features += large_numbers

    constraint_keywords = [
        "constraint", "constraints", "at most",
        "no more than", "less than", "greater than"
    ]
    keyword_count = sum(text.count(k) for k in constraint_keywords)
    features += keyword_count

    inequality_count = sum(text.count(sym) for sym in ["<", ">", "≤", "≥"])
    features += inequality_count

    return features
```

Models Used & Experimental Setup:

LinearSVC: The classification task (difficulty level) was done using a Linear Support Vector Classifier (LinearSVC), which is well-suited for high-dimensional sparse text features. Balanced class weights were used because in the dataset, there were more hard questions (1941) than easy ones (766).

// There was a mismatch in the score and class when I used both the models for the UI. So, in the end I simply used the score from regressor to assign the class (if score < 3.33, classify as “easy” and so on) //

Linear Regression: For numerical difficulty prediction, Linear Regression was used as a baseline regression model. Despite its simplicity, it achieved strong performance, indicating that difficulty scores follow a relatively linear trend with respect to extracted features.

Results

The classification model achieved an accuracy of approximately 48%, which is better than random guessing (33%). The model performed best on hard problems (56% precision), while medium problems were the most challenging to classify.

The regression model achieved a Mean Squared Error (MSE) of 12.92 corresponding to an RMSE of approx 3.59.

```
Aarav@Aarav MINGW64 ~/OneDrive/Coding_/WebDev/AutoJudge (main)
$ ./venv/Scripts/python ./src/classifier.py
precision    recall  f1-score   support

   easy      0.40      0.41      0.40      153
   hard      0.56      0.61      0.58      389
  medium      0.38      0.34      0.36      281

 accuracy      0.48      823
 macro avg      0.45      0.45      0.45      823
weighted avg      0.47      0.48      0.47      823
```



```
Aarav@Aarav MINGW64 ~/OneDrive/Coding_/WebDev/AutoJudge (main)
$ ./venv/Scripts/python ./src/regression.py
MSE: 12.929295363912711
```

Web Interface:

The interface allows users to paste a problem description, input format, and output format. When the user presses submit, the backend processes the input using the trained models and displays the predicted difficulty class and score.

AutoJudge

Paste a programming problem below:

Problem Description

```
{ "title": "Unicyclic Count", "description": "A unicyclic graph is a graph with\n exactly one cycle. A spanning subgraph\n of a graph $G$ is a\n subgraph that has one component and includes all the vertices\n of $G$. Given a simple\n graph $G$, count the\n number of spanning unicyclic subgraphs. The illustration below\n shows the visualization of Sample Input/Output $15$.\n"
```

Input Description

```
"input_description": "The first line of the input contains two integers,\n $V$ and $E$, representing the number of\n vertices and edges of the graph $G$\n respectively. ($1 \le V \le 17, 0 \le E \le \lfloor V(V-1)/2 \rfloor$)"
```

Output Description

```
"output_description": "Output one integer, representing the number of spanning\n unicyclic subgraphs. As the number can be rather big, output it\n modulo $10^9 + 7$."
```

Predict

Predicted Difficulty: hard

Predicted Score: 7.41

AutoJudge

Paste a programming problem below:

Problem Description

```
"Congratulations, you have been accepted to the Master's Assistance Center! However, you were extremely bored in class and got tired of doing nothing, so you came up with a game for yourself.\n\nYou are given a string s\n and an even integer n\n . There are two types of operations that you can apply to it:\n\nAdd the reversed string s\n to the end of the string s\n (for example, if s =
```

Input Description

```
""
```

Output Description

```
""
```

Predict

Predicted Difficulty: easy

Predicted Score: 2.66

Video : https://drive.google.com/drive/folders/1AjzuTpLAldy1cqSpjao5PHv_28UNFij6?usp=drive_link